

Assignment 2 - Part 2: Multithreading with pthreads

Task 3: Benchmarks

All times are in seconds (run in WSL Ubuntu, 20 cores).

Array Sum Problem (for array size 100,000,000)				
Sequential	1 thread	2 threads	3 threads	4 threads
0.209994	0.194991	0.084209	0.058479	0.042271

Matrix Multiplication Problem (for matrix size 1,000)				
Sequential	1 thread	2 threads	3 threads	4 threads
2.888038	1.775123	0.872184	0.571042	0.434687

Task 4: Observations

Adding threads made both programs faster. The array sum went from 0.195 s with 1 thread to 0.042 s with 4 threads, and the matrix multiply went from 1.78 s to 0.43 s. That's about 4x faster with 4 threads, so it's close to perfect speedup. My computer has 20 cores, so each thread got its own core.

The sequential matrix code (2.89 s) was slower than 1 thread (1.78 s) because it adds straight into `C[i][j]` every time, while my thread code adds into a local variable first.

I didn't need synchronization. Each thread only wrote to its own spot (its own `partialSums` slot, or its own rows of `C`), so no two threads changed the same data. I only used `pthread_join` so the main thread waits until all the threads finish before using the results. A mutex would be needed if all the threads added into one shared total.

The manual lock won't work. Checking `lock == 0` and setting `lock = 1` are two separate steps, so two threads can both see 0 and both enter the critical section. A real lock like a mutex does the check and set in one step.

Screenshot: code works as expected

```
mason@Mason:~/A2_part_2$ ./sumT_4
Threads: 4
Total Sum: 5000000050000000
Time: 0.052922 seconds
mason@Mason:~/A2_part_2$ ./matrixT_4
Matrices initialized successfully.
Matrix multiplication complete!
Threads: 4
Result check: CORRECT (C[0][0] = 1000, C[999][999] = 1000)
Time: 0.440277 seconds
mason@Mason:~/A2_part_2$ █
```